JAVA PROGRAMMING ASSIGNMENT 1

CAMPUS SMART-PARKING SYSTEM

Submitted by	Madaraboina Gayathri
Register number	192472074
Programme	B.Tech
Department	Computer Science and Engineering (CSE – AI)
Institution	SIMATS Engineering
Assignment	Programming in Java – Assignment 1
GitHub repository	CSA0918-192472074-Assignment1

ACADEMIC PROJECT REPORT

Submitted in partial fulfilment of the requirements for the Java Programming assignment.

ABSTRACT

This report presents the design, implementation and validation of a Campus Smart-Parking System using Java and object-oriented programming. The system models vehicles, parking spaces, parking tickets and visitor passes. The work progresses through twelve assignment tasks, from problem analysis and Java fundamentals to inheritance, polymorphism, abstraction, interfaces, encapsulation and an integrated multi-floor application.

The final implementation supports cars, two-wheelers, electric bikes and delivery vans. It allocates vehicles to the first available space across five floors, calculates vehicle-specific parking fees, records ticket details, removes vehicles by registration number and validates a visitor pass for its issue date.

Keywords: Java, object-oriented programming, inheritance, polymorphism, abstraction, interfaces, parking allocation and parking ticket.

TABLE OF CONTENTS

• 1. Introduction
• 2. Problem Statement and Objectives
• 3. Scope and Requirements
• 4. System Design
• 5. Task-wise Implementation Summary
• 6. Algorithms and Pseudocode
• 7. Test Plan and Results
• 8. Complexity Analysis
• 9. Validation and Limitations
• 10. Conclusion and Future Enhancements
• 11. References
• Appendix – Execution Screenshots

1. INTRODUCTION

Campus parking requires a consistent way to identify vehicles, allocate spaces, calculate charges and maintain records of entry and exit. Manual processes can lead to allocation conflicts, incorrect fee calculations and difficulty tracking occupancy. This Java console application demonstrates how object-oriented modelling can organise these responsibilities into reusable classes.

2. PROBLEM STATEMENT AND OBJECTIVES

Problem statement: Design and implement a Java-based campus parking model that stores vehicle details, allocates and releases parking spaces, calculates charges based on vehicle type and duration, issues parking tickets and supports visitor-pass validation.

- Identify suitable classes, data and operations for a parking facility.
- Apply Java data types, arrays, constructors, static members and object references.
- Demonstrate inheritance, overriding, runtime polymorphism, abstraction and interfaces.
- Protect object state using encapsulation and access modifiers.
- Allocate vehicles across five floors and record ticket details.
- Apply different fee rules to cars, two-wheelers, electric bikes and delivery vans.
- Validate a one-day visitor pass and test the completed application.

3. SCOPE AND REQUIREMENTS

3.1 Functional Requirements

- The facility contains five floors with capacities 2, 3, 4, 2 and 5 spaces (16 total).
- The allocator searches floors and slots in order and selects the first free space.
- A ticket records ticket number, vehicle, duration, fee, floor and slot.
- A vehicle can be removed using its registration number.
- The system displays occupied spaces and issued tickets.
- Cars are charged ■30 per started hour; two-wheelers ■10 per started hour.
- Electric bikes are charged a flat ■25 for a valid duration.
- Delivery vans are charged twice the car rate from 09:00 inclusive to 17:00 exclusive; otherwise the normal car rate applies.
- A visitor pass is valid on its issue date only.

3.2 Non-functional Requirements

- Readable, modular Java classes with clear responsibilities.
- Clear console output for allocation, occupancy, tickets and fees.
- Validation of facility configuration and parking duration.
- A design that can be extended with additional vehicle types.

4. SYSTEM DESIGN

4.1 Class Responsibilities

Class	Responsibility
Vehicle	Abstract base class; stores registration number and defines fee calculation.
Car	Car fee rule: ■30 per started hour.
TwoWheeler	Two-wheeler fee rule: ■10 per started hour.
ElectricBike	Electric-bike fee rule: flat ■25.
DeliveryVan	Specialised car with time-dependent double-rate rule.
ParkingFacility	Manages five floors, slot allocation, removal, occupancy and tickets.
ParkingTicket	Stores ticket number, vehicle, duration, fee, floor and slot.
VisitorPass	Stores pass number, visitor name and issue date; validates same-day use.
Main	Creates sample objects and demonstrates system operations.

4.2 Textual UML View

```
Vehicle (abstract)
registration number; fee calculation
/ | \
/ | \
Car TwoWheeler ElectricBike
|
DeliveryVan

ParkingFacility -- manages --> Vehicle
ParkingFacility -- creates --> ParkingTicket
VisitorPass -- validates --> issue date
Main -- demonstrates all components
```

The Vehicle base type provides a common reference for different vehicle subclasses. ParkingFacility can therefore manage multiple vehicle types without needing a separate allocation algorithm for each type.

4.3 Floor Capacities

Floor	Capacity
1	2
2	3
3	4
4	2
5	5
Total	16

5. TASK-WISE IMPLEMENTATION SUMMARY

Task 01 – Problem Analysis

Identified the parking problem, inputs, outputs, candidate classes, responsibilities and business rules; compared object-based modelling with parallel arrays.

Task 02 – Java Data Representation

Selected suitable primitive and reference types, considered arrays and jagged arrays, and documented input handling and numeric conversion concerns.

Task 03 – Array-based Parking Facility

Implemented allocation and release using an array-based facility. Allocation searches for a free slot and release clears an occupied slot.

Task 04 – Classes and Constructors

Covered class responsibilities, overloaded constructors, this, constructor chaining, copy-constructor concepts and object memory.

Task 05 – Static Members

Applied shared static state for ticket numbering and reviewed static methods, static blocks, object lifecycle and object references.

Task 06 – Inheritance

Introduced a vehicle hierarchy and specialised subclasses, using super to initialise inherited state.

Task 07 – Method Overriding

Applied overriding and runtime polymorphism, @Override, final, toString, equals and hashCode.

Task 08 – Abstract Classes and Interfaces

Demonstrated an abstract Vehicle fee model, a Chargeable interface, runtime dispatch and equality based on registration number.

Task 09 – Abstraction and Interfaces

Explained how abstraction hides implementation details and how interfaces define capabilities.

Task 10 – Packages and Encapsulation

Organised classes into packages, used access modifiers and private fields, and reviewed wrapper classes, autoboxing and equality comparisons.

Task 11 – Integrated Parking Management

Integrated vehicles, tickets and facility operations: parking, listing, ticket display, fee summary and vehicle removal.

Task 12 – Extended Smart Parking

Added five floors, electric bikes, delivery vans, visitor passes, floor/slot ticket details and vehicle-specific fee rules.

6. ALGORITHMS AND PSEUDOCODE

6.1 Allocate a Vehicle

```
ALLOCATE(vehicle, duration, entryTime)
validate vehicle and duration
for each floor from 1 to 5
for each slot on that floor
if slot is empty
calculate fee using vehicle fee rule
store vehicle in slot
create ticket with floor, slot, duration and fee
return allocation details
report that no space is available
```

6.2 Remove a Vehicle

```
REMOVE(registrationNumber)
for each floor
for each slot
if slot contains matching registration number
clear the slot
return success
return not found
```

6.3 Fee Calculation Rules

```
STARTED_HOURS(minutes) = ceiling(minutes / 60)
CAR_FEE = STARTED_HOURS(minutes) * 30
TWO_WHEELER_FEE = STARTED_HOURS(minutes) * 10
ELECTRIC_BIKE_FEE = 25 for valid duration
DELIVERY_VAN_FEE:
base = CAR_FEE
if 09:00 <= entryTime < 17:00: return 2 * base
otherwise: return base
VISITOR_PASS_VALID = (today equals issueDate)
```

The pseudocode summarises the implemented rules; refer to the submitted Java source files for exact method signatures and validation messages.

7. TEST PLAN AND RESULTS

The following test cases are based on the successful Task 12 console output and the earlier Task 11 execution output.

Test	Input/action	Expected behaviour	Observed result
T01	Car, 120 min, 10:00	2 × ■30 = ■60	Pass – ■60
T02	Two-wheeler, 60 min	1 × ■10 = ■10	Pass – ■10
T03	Electric bike, 180 min	Flat fee ■25	Pass – ■25
T04	Delivery van, 120 min, 10:00	Double car fee = ■120	Pass – ■120
T05	Allocate four vehicles	First available spaces in order	Pass – floors 1 and 2
T06	Visitor pass on issue date	Pass is valid	Pass – true
T07	Remove by registration number	Matching slot is cleared	Pass – bike removed in Task 11

The actual screenshots supplied are included in Appendix A. They show the outputs for Tasks 03, 08, 11 and 12.

8. COMPLEXITY ANALYSIS

Let S be the total number of parking spaces and T the number of issued tickets. The current design uses sequential searches over the slot arrays.

Operation	Time complexity	Reason
Allocate first free slot	O(S)	May inspect every slot.
Remove by registration number	O(S)	May search all slots.
Display occupancy	O(S)	Checks each slot.
Display all tickets	O(T)	Visits each ticket.
Fee calculation	O(1)	Fixed arithmetic and time comparisons.
Visitor pass validation	O(1)	One date comparison.

The facility slot structure requires O(S) space. Ticket storage grows with the number of parking events and requires O(T) space.

9. VALIDATION AND LIMITATIONS

The successful runs demonstrate the main workflows. Additional tests recommended before final submission include a completely full facility, allocation when no space remains, removal of an unknown registration number, duplicate registration numbers, zero and negative durations, delivery-van entry exactly at 09:00 and 17:00, and visitor-pass validation on the day after issue.

This is a console-based educational model. It does not persist data after program termination, authenticate staff, integrate payment services, handle concurrent entry gates or connect to physical occupancy sensors. The delivery-van rule is based on entry time; charging only for the portion of a stay overlapping the daytime window would require a more advanced calculation.

10. CONCLUSION AND FUTURE ENHANCEMENTS

The Campus Smart-Parking System demonstrates core Java and object-oriented programming concepts in a practical campus scenario. The twelve tasks progress from analysis and data representation to constructors, static members, inheritance, overriding, abstraction, interfaces, encapsulation and an integrated multi-floor application. The final run demonstrates allocation, fee calculation, ticket generation and visitor-pass validation.

Future Enhancements

- Persist vehicle and ticket records using a database.
- Develop a graphical or web-based interface for drivers and administrators.
- Add indexed lookup to improve search performance in larger facilities.
- Integrate QR-code tickets, digital payments and automated barriers.

- Connect sensors or cameras to show live parking availability.
- Add user roles, audit logs and stronger validation.

11. REFERENCES

- Oracle, Java Platform, Standard Edition Documentation: https://docs.oracle.com/en/java/
- Oracle, The Java Tutorials – Object-Oriented Programming Concepts: https://docs.oracle.com/javase/tutorial/java/concepts/
- Course assignment brief: Programming in Java – Assignment 1, Campus Smart-Parking System.
- Project Java source files and recorded console execution outputs.

APPENDIX A – EXECUTION SCREENSHOTS

The following screenshots were supplied as evidence of successful program execution.

Task 03 – Array-based slot allocation and release

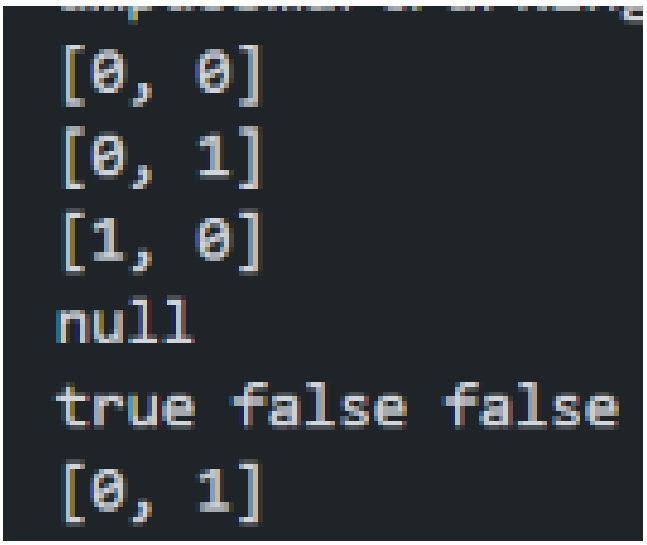


Figure A1. Task 03 output showing allocated slot positions, null for no available allocation, and release status.

Task 08 – Fee calculation and polymorphism

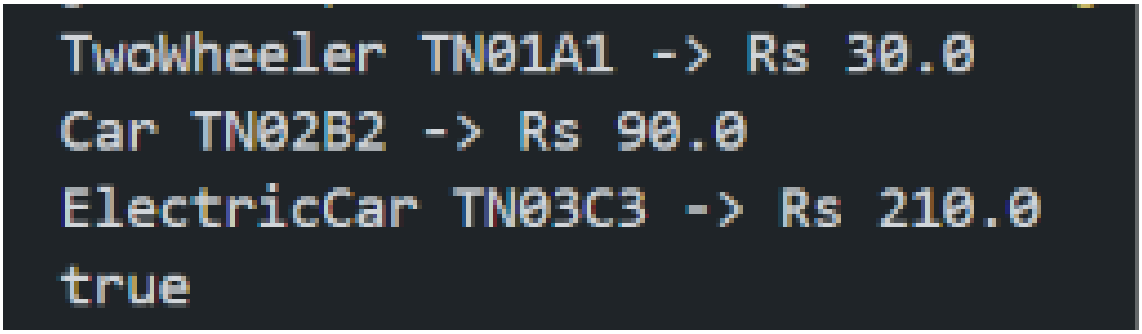


Figure A2. Task 08 output showing vehicle-specific fees and equality result.

Task 11 – Integrated parking management

```
PS C:\Users\Madaraboina Gayathri\OneDrive\Desktop\java\CampusSmartParking\TASK11> java Main
Vehicle parked in slot 1
Vehicle parked in slot 2

--- Parked Vehicles ---
Slot 1: TwoWheeler [TN01A1234]
Slot 2: Car [TN02B5678]

--- Parking Tickets ---
Ticket #1001 | TwoWheeler [TN01A1234] | Minutes: 60 | Fee: Rs 10.0
Ticket #1002 | Car [TN02B5678] | Minutes: 130 | Fee: Rs 90.0

--- Fee Summary ---
Bike fee: Rs 10.0
Car fee: Rs 90.0
Vehicle removed from slot 1

--- Parked Vehicles ---
Slot 2: Car [TN02B5678]
```

Figure A3. Task 11 output showing parked vehicles, tickets, fee summary and vehicle removal.

Task 12 – Five-floor Campus Smart-Parking System

```
===== CAMPUS SMART PARKING =====
Parked on floor 1, slot 1
Parked on floor 1, slot 2
Parked on floor 2, slot 1
Parked on floor 2, slot 2

--- Parked Vehicles ---
Floor 1:
  Slot 1: Car [TN01A1234]
  Slot 2: TwoWheeler [TN02B5678]
Floor 2:
  Slot 1: ElectricBike [TN03C9012]
  Slot 2: DeliveryVan [TN04D3456]
Floor 3:
  No vehicles parked.
Floor 4:
  No vehicles parked.
Floor 5:
  No vehicles parked.

--- Parking Tickets ---
Ticket #1001 | Car [TN01A1234] | Minutes: 120 | Floor: 1 | Slot: 1 | Fee: Rs 60.0
Ticket #1002 | TwoWheeler [TN02B5678] | Minutes: 60 | Floor: 1 | Slot: 2 | Fee: Rs 10.0
Ticket #1003 | ElectricBike [TN03C9012] | Minutes: 180 | Floor: 2 | Slot: 1 | Fee: Rs 25.0
Ticket #1004 | DeliveryVan [TN04D3456] | Minutes: 120 | Floor: 2 | Slot: 2 | Fee: Rs 120.0

===== VISITOR PASS =====
Visitor Pass #5001 | Name: Campus Visitor | Valid on: 2026-10-30
Valid today: true
```

Figure A4. Task 12 output showing five-floor allocation, vehicle-specific tickets and visitor-pass validation.

APPENDIX B – SUBMISSION CHECKLIST

- Include TASK01 through TASK12 folders in the GitHub repository.
- Keep source .java files and task notes in their corresponding folders.
- Exclude generated .class files from version control using *.class in .gitignore.
- Include this final PDF in the report folder.
- Verify that the GitHub repository is accessible to the evaluator.
- Make task-wise commits if required by the assignment brief.